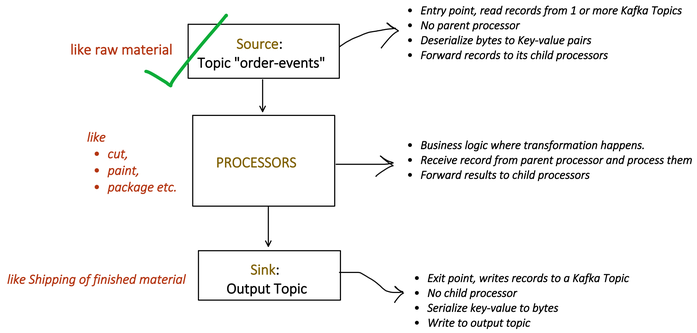
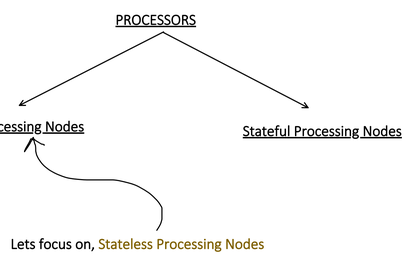
Kafka Stream: Stateless Processing Nodes

Tuesday, 21 July 2026 1:59 PM

TOPOLOGY has 3 parts:

- Source
- Processor
- Sink





- A stateless node processes each record independently.
- It does not remember anything from previous records.

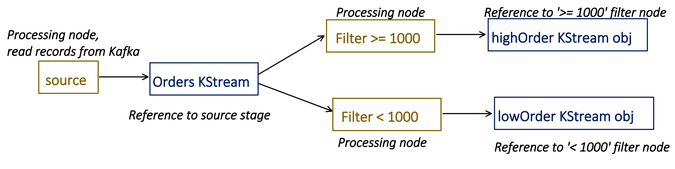
Important Stateless Operations:

We will discuss these stateless operations in depth:

1. filter
2. map
3. mapValues
4. selectKey
5. split / branch
6. merge

Etc.

Generally Stateless operations returns KStream.



Think of KStream as:

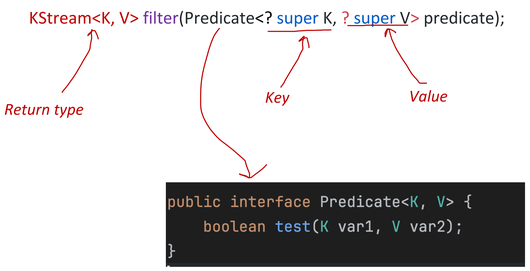
- A KStream represents a stage in the processing pipeline. It does not contain data.

- Multiple KStream objects allow us to continue building the topology from different stages of the same pipeline.

1. filter

filter keeps only the records that satisfy a condition.

- If the condition returns true, the record continues in the stream.
- If the condition returns false, the record is dropped.



Example:

```
@Configuration
public class SourceTypesTopology {

    @Bean
    public void kstreamDemo(StreamsBuilder builder) {


            Serde<Order> orderSerde = new JsonSerde<>(Order.class);

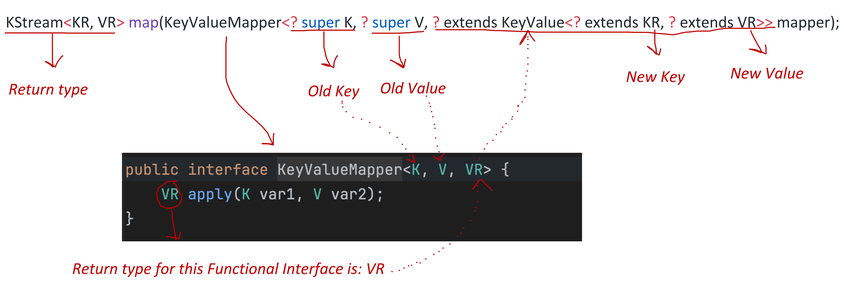
            //Processing node: Reading from source
            //orders KStream act as reference to source processing nodes.
            KStream<String, Order> orders = builder.stream("order-events",
                                                           Consumed.with(Serdes.String(), orderSerde));


            //Processing node: filter amount > 1000
            //highValueOrders KStream act as reference to filter processing nodes.
            KStream<String, Order> highValueOrders =
                                                orders.filter((key, order) -> order.getTotalAmount() > 1000)
                    .to("order-events-output",
                                                Produced.with(Serdes.String(), orderSerde));
    }
}
```

2. map

Map transforms the records into another record.
It can change:

- only Key
- only Value
- or Both Key and Value



```
@Configuration
public class SourceTypesTopology {

    @Bean
    public void kstreamDemo(StreamsBuilder builder) {


            Serde<Order> orderSerde = new JsonSerde<>(Order.class);

            //Processing node: Reading from source
            //orders KStream act as reference to source processing nodes.
            KStream<String, Order> orders = builder.stream("order-events",
                                                    Consumed.with(Serdes.String(), orderSerde));


            //Processing node: map, change only the key
            //updatedOrders KStream act as reference to this map processing nodes.
            KStream<String, Order> updatedOrders=
                                    orders.map((key, order) -> KeyValue.pair("ORDER-" + key, order));

            updatedOrders.to("order-events-output",
                                        Produced.with(Serdes.String(), orderSerde));
    }
}
```

> If the input record is:
> ("101", Order)
>
> After map() it becomes:
> ("ORDER-101", Order)

Like map() is used to change either Key or Value or Both.

Similarly we have:

3. mapValues():
    - No key change is allowed
    - Only Value can be changed

```
KStream<String, OrderAlert> alerts = orders.mapValues(order -> new OrderAlert(order.getOrderId(), order.getAmount()));
```

Input:

Key	Value
order-1	Order

Output:

Key	Value
order-1	OrderAlert

4. selectKey():
    - Only Key change is allowed
    - No Value change is allowed

```
KStream<String, Order> ordersByUser = orders.selectKey((orderId, order) -> order.getUserId());
```

Input:

Key	Value
order-1	{ userId: "u1", amount: 500 }

Output:

Key	Value
u1	{ userId: "u1", amount: 500 }

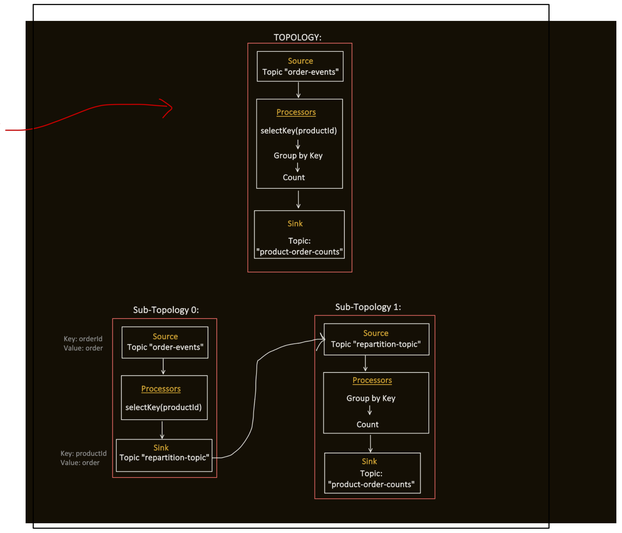
Method	Can change key	Can change value	May require repartition later
map()	Yes	Yes	Yes (if key is changed)
mapValues()	No	Yes	No
selectKey()	Yes	No	Yes

Note:

When key is changed, it does not initiate the repartition immediately.
Repartition initiated when downstream operation require records to be partitioned by the new key.

Generally its done by:
- groupByKey
- join

I explained repartition in Kafka Stream Architecture part, kindly check it out, if there is any doubt with this term



5. Split / Branch

Problem with below code:

When we have 2 filters:

```
//Processing node: filter amount > 1000
//highValueOrders KStream act as reference to filter processing nodes.
KStream<String, Order> highValueOrders =
                                        orders.filter((key, order) -> order.getTotalAmount() > 1000));


//Processing node: filter amount <= 1000
//lowValueOrders KStream act as reference to filter processing nodes.
KStream<String, Order> lowValueOrders =
                                        orders.filter((key, order) -> order.getTotalAmount() <= 1000));
```

For each record, both the filters will get executed.

So above approach is okay, when a record can move in both the direction, else why for every record, we need both the filters to be checked.

Then we can use split/branch.

```
Map<String, KStream<String, Order>> branches = orders.split()
                                                    .branch((k, o) -> o.getTotalAmount() >
                                                    2000,Branched.as("premium"))
                                                    .branch((k, o) -> o.getTotalAmount() > 1000,Branched.as("high"))
                                                    .defaultBranch(Branched.as("low"));

KStream<String, Order> premiumOrders = branches.get("premium");
KStream<String, Order> highOrders = branches.get("high");
KStream<String, Order> lowOrders = branches.get("low");
```

So its like 'if-else if - else' chain. Not all conditions will get executed.

6. Merge

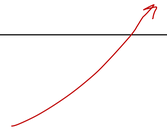
merge() is the opposite of split().

- **split()** -> One stream becomes many.
- **merge()** -> Many streams become one.

```
Map<String, KStream<String, Order>> branches = orders.split()
                                                      .branch((k, o) -> o.getTotalAmount() >
                                                      2000,Branched.as("premium"))
                                                      .branch((k, o) -> o.getTotalAmount() >
                                                      1000,Branched.as("high"))
                                                     .defaultBranch(Branched.as("low"));

KStream<String, Order> premiumOrders = branches.get("premium");
KStream<String, Order> highOrders = branches.get("high");
KStream<String, Order> lowOrders = branches.get("low");

KStream<String, Order> allOrders = premiumOrders
                                          .merge(highOrders)
                                          .merge(lowOrders);
```



Key and order type should be same for all